

Examen final

Carlos Agon

Déroulement de L'examen

Tous les documents sont autorisés, et notamment ceux du cours.

L'utilisation de tout moyen de communication (téléphone, tablette, ordinateur personnel, etc.) est interdite.

L'utilisation d'une clé ou disque USB personnel, en lecture seule et étiquetée à votre nom, est autorisée; vous ne devez pas le prêter à un camarade.

Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction.

L'examen dure 2h.

Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

Le langage à étendre est ILP4. Le paquetage Java correspondant sera nommé `com.paracamplus.ilp4.ilp4examen`. Sera ramassé, à partir de votre `~/workspace` (situé sous ce nom directement dans votre HOME), le **seul** répertoire `~/workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/ilp4examen/` et tous ses sous-répertoires.

Pour vous éviter de la taper à nouveau, voici l'url du site du master (mais il faut, pour y accéder, ne plus passer par le proxy) et celle de l'ARI où se trouvent de nombreuses documentations dont celle de Java :

<http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/>
<http://www-ari.ufr-info-p6.jussieu.fr/>

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve.

Mise en route

- Ouvrez un navigateur et allez à l'url :<https://www-master.ufr-info-p6.jussieu.fr:8083/2017/DLP>. Vous y trouverez les sources d'ilp ainsi qu'une version pdf de ce sujet,
- Ouvrez eclipse
- Choisissez le repertoire `workspace` comme `workspace`
- Extraire l'archive dans le repertoire `workspace` (vous devez obtenir le repertoire `~/workspace/ILP-UPMC`)

- Créez un nouveau projet Java en cliquant sur : File -> New -> Java Project.
- Choisissez ILP-UPMC en project name (en bas de la fenetre devrait s'afficher le message : "the wizard will automatically ...").
- Cliquer sur Next
- (facultatif, puisqu'on ne demande pas le code du compilateur) Compilez la bibliothèque d'exécution en faisant :

```
cd ~/workspace/ILP-UPMC/C/; make
```

- le plugin ANTLR devrait fonctionner sans réglages particuliers. Toutefois, si ce n'est pas le cas, vous pouvez toujours utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
cd ~/workspace/ILP-UPMC/ANTLRGrammars
java -jar ~/workspace/ILP-UPMC/Java/jars/antlr-4.4-complete.jar \
-lib ~/workspace/ILP-UPMC/ANTLRGrammars \
-o ~/workspace/ILP-UPMC/target/generated-sources/antlr4/ \
LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

On rappelle que l'on peut aussi lancer le plug-in à la main depuis Eclipse avec "Run as..." si il ne se lance pas automatiquement lors de l'édition d'une grammaire.

Troubleshooting

- Si jamais eclipse se bloque en essayant de se connecter à svn, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez eclipse.

Introduction

Dans cet examen, nous voulons étendre ILP4 et y inclure l'expression `super_with_args`. L'expression `super_with_args`, à l'intérieur d'une méthode, permet d'invoquer la méthode `super` mais en donnant de nouveaux arguments.

Commençons par rappeler le fonctionnement de `super` grâce à l'exemple suivant :

```
class Point extends Object
{
    var x;

    method m1 (t)
        self.x + t;
}

class Point2D extends Point
{
    var y;

    method m1 (t)
        self.y + super;
}

let pc = new Point2D(4, 5) in
    1 + pc.m1(2)
```

Le résultat du programme précédent est 12 car `pc.m1(2) = 5 + super`. `super` appelle la méthode `m1` définie dans la classe `Point` avec `t = 2`. La réponse est donc `4 + 2 = 6`.

Changeons maintenant la définition de la méthode `m1` dans la classe `Point2D` ainsi :

```
class Point2D extends Point
{
    var y;

    method m1 (t)
        self.y + super_with_args(3);
}
```

Le résultat du programme est cette fois-ci 13 car `super_with_args(3)` appelle la méthode `m1` définie dans la classe `Point` mais avec `t = 3`. La réponse est donc $4 + 3 = 7$.

1 Echauffement (1 point)

Avec cette nouvelle définition de la class `Point2D` :

```
class Point2D extends Point
{
    var y;

    method m1 (t)
        self.y + super_with_args(super);
}
```

Quel est le résultat de l'expression suivante ?

```
let pc = new Point2D(4, 5) in
    1 + pc.m1(2)
```

Ecrivez votre réponse dans un fichier nommé `com/paracampus/ilp4/ilp4examen/question1.txt`.
Si vous croyez que l'appel `super_with_args(super)` n'est pas possible, écrivez simplement "ERREUR"

2 Grammaire (2 points)

Ecrivez une grammaire définissant cette nouvelle extension nommée `com/paracampus/ilp4/ilp4examen/grammars/ILPMLgrammarExamen.g4`.

3 AST (3 points)

Proposez une extension de l'arbre syntaxique avec la nouvelle expression `super_with_args`. On rappelle que vous devez fournir les classes AST ainsi que les interfaces correspondantes dans `com.paracampus.ilp4.ilp4examen.ast` et `com.paracampus.ilp4.ilp4examen.interfaces`. On n'oubliera pas d'inclure la classe `ASTfactory` correspondante et son interface `IASTfactory` ainsi que la nouvelle interface `IASTvisitor`.

4 Analyse syntaxique (2 points)

Ecrivez dans le package `com.paracampus.ilp4.ilp4examen.parser` les nouvelles classes `ILPMLparser` et `ILPMLlistener` permettant de construire les nouveaux noeuds ajoutés au langage.

5 Interpétation (5 points)

Implantez le traitement des nouveaux noeuds `super_with_args` dans une classe `com.paracamplus.ilp4.ilp4examen.interpreter.Interpreter.java`.

6 Compilation (7 points)

6.1 Questions (2 points)

Répondez aux questions suivantes :

- listez les classes et interfaces que vous auriez à ajouter pour mettre en œuvre le compilateur, en précisant pour chacune les classes et interfaces qu'elles étendent ou implantent ;
- précisez quels aspects de l'instruction `super_with_args` et de son exécution sont statiques, et lesquels sont dynamiques

6.2 Schéma de compilation (5 points)

Proposez un schéma de compilation pour l'expression `super_with_args` **ou** écrivez le code pour la méthode

```
@Override
public Void visit(IASTsuperwithargs iast, Context context)
    throws CompilationException {
    ...
}
```

dans un fichier `com.paracamplus.ilp4.ilp4examen.compiler.Compiler.java`

7 Bonus : Tests (2 points)

Créez un répertoire `com/paracamplus/ilp4/ilp4examen/SamplesExamen/` dans lequel vous mettrez un programme ILP illustrant un exemple intéressant d'utilisation du `super_with_args`. Vous fournirez bien sûr les 3 fichiers : `.ilpml`, `.print` et `.result`. Testez votre implantation sur votre programme en fournissant la classe de test `com.paracamplus.ilp4.ilp4examen.interpreter.InterpreterTest.java`. Elle devra être compatible avec les précédentes versions du langage.